

ChatWave

Complete Project Documentation

A full, beginner-friendly guide to the Real-Time Chat Application —
frontend, backend, Redux, database, real-time messaging, media uploads,
with the definition of every concept used.

MERN Stack · React · Redux Toolkit · Node/Express · MongoDB · Socket.IO · Cloudinary

Table of Contents

1. Introduction
2. Technology Stack (with definitions)
3. High-Level Architecture
4. Project Folder Structure
5. Frontend — Entry & Routing
6. Frontend — Pages
7. Frontend — Components
8. Frontend — Utilities & Hooks
9. Redux State Management (full flow)
10. Backend — Server & Middleware
11. Backend — Routes & Controllers
12. Database Design (models & relationships)
13. Authentication & Security
14. Real-Time Messaging (Socket.IO)
15. Media Uploads (images & files / Cloudinary)
16. Complete API Reference
17. End-to-End Data Flows
18. Feature Walkthroughs
19. Environment Variables
20. Deployment Guide
21. Annotated Source Code (frontend & backend)
22. Glossary (every term defined)
23. UI Gallery (screenshots)
24. FAQ — common questions
25. Getting Started — Setup & Run
26. Testing the Application
27. Troubleshooting & Common Errors
28. Best Practices & Future Improvements
29. Appendix — Component Reference
30. Visual Sequence Diagrams

1. Introduction

ChatWave is a real-time one-to-one chat application built on the **MERN** stack (MongoDB, Express, React, Node.js) with **Socket.IO** for live messaging. Users can register, log in, see other users, chat in real time, send images and files, manage their profile, and control whether others can see their online status.

What you can do in the app

- **Register & log in** securely (passwords hashed, sessions via cookies).
- **See all other users** with live online/offline presence.
- **Chat in real time** — messages appear instantly without refreshing.
- **Send images and documents** as attachments.
- **Manage your profile** — change name, gender, photo, and status visibility.
- **Stay logged in** across refreshes (state is persisted).

Who this document is for

This guide is written for beginners. Every technical term is explained in plain English the first time it appears, and again in the **Glossary** (Section 21). Read it top to bottom, or jump to a section using the Table of Contents.

The two halves of the app

Half	Folder	Job
Frontend (client)	frontend/	What the user sees and clicks (React).
Backend (server)	backend/	Logic, database, and real-time engine (Node/Express).

They talk to each other over **HTTP** (normal requests) and a **WebSocket** (live connection).

2. Technology Stack (with definitions)

Here is every major technology used, what it *is*, and why this project uses it.

Frontend technologies

Technology	What it is	Why we use it
React	A JavaScript library for building user interfaces from reusable components.	Builds the whole UI.
Create React App	A ready-made React project setup and build tool.	Runs & bundles the app.
Redux Toolkit	The official, simple way to use Redux — a central state container.	Shares data app-wide.
redux-persist	Saves the Redux state to browser storage and restores it.	Keeps you logged in.
React Router	Handles navigation between pages without reloading.	Login / Signup / Home routes.
Tailwind CSS	A utility-class CSS framework for styling.	All the styling.
daisyUI	Prebuilt UI components on top of Tailwind.	Some base UI pieces.
Axios	A library to make HTTP requests.	Calls the backend APIs.
socket.io-client	The browser side of Socket.IO.	Receives live messages.
react-hot-toast	Small popup notifications (toasts).	Success / error messages.
react-icons	A big icon set as React components.	All the icons.

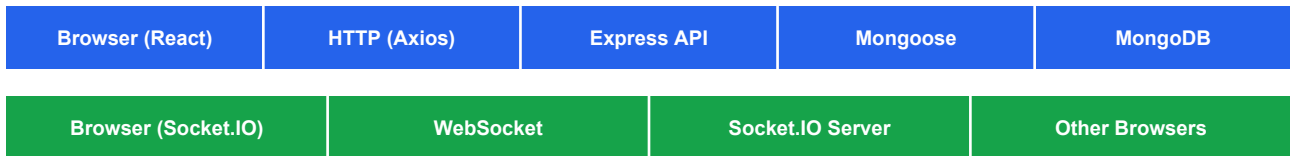
Backend technologies

Technology	What it is	Why we use it
Node.js	A runtime that lets JavaScript run on a server.	Runs the backend.
Express	A minimal web framework for Node.	Routes & HTTP server.
MongoDB	A NoSQL document database (stores JSON-like data).	Stores users & messages.
Mongoose	An ODM that models MongoDB data with schemas.	Defines & queries data.
JWT	JSON Web Token — a signed token proving identity.	Login sessions.
bcryptjs	A library to hash (scramble) passwords.	Secure passwords.
cookie-parser	Reads cookies from incoming requests.	Reads the auth cookie.
cors	Controls which websites may call the API.	Allows the frontend.
Socket.IO	Real-time, two-way communication over WebSockets.	Live messaging & presence.

Technology	What it is	Why we use it
Cloudinary	Cloud media hosting & delivery.	Stores uploaded images/files.
dotenv	Loads secrets from a .env file into the app.	Keys & config.

3. High-Level Architecture

At the highest level, data flows like this:



How the pieces cooperate

- The **React** app renders the UI and holds shared data in the **Redux** store.
- For normal actions (login, fetch users, send a message), it calls the **Express** API with **Axios**.
- Express runs **middleware** (security checks), then a **controller** (logic), which uses **Mongoose** to read/write **MongoDB**.
- For live updates, a **Socket.IO** connection stays open; the server pushes new messages and the online-user list instantly.
- Uploaded media is sent to **Cloudinary** and only its URL is stored in the database.

Think of HTTP as sending letters (request → reply), and the WebSocket as a phone call that stays connected so either side can talk at any time.

4. Project Folder Structure

Frontend (frontend/src)

```
src/  
  index.js      # app entry; BASE_URL; Redux Provider + PersistGate  
  App.js        # routes (/, /login, /signup); socket setup  
  index.css     # global styles (Tailwind)  
  components/   # all UI pieces (see Section 7)  
  redux/        # store + slices (user, message, socket)  
  hooks/        # data-fetching hooks  
  utils/        # avatar + helpers
```

Backend (backend/)

```
backend/  
  index.js      # server start, middleware, mount routes  
  config/       # database.js (Mongo), cloudinary.js (uploads)  
  routes/       # userRoute.js, messageRoute.js  
  controllers/  # userController.js, messageController.js  
  middleware/   # isAuthenticated.js (JWT check)  
  models/       # userModel, messageModel, conversationModel  
  socket/       # socket.js (Socket.IO server)
```

Each folder has one clear job. This separation (routes vs controllers vs models) is called a **layered architecture** and keeps code easy to find and change.

5. Frontend — Entry & Routing

index.js — the starting point

This file mounts React into the page and wraps the app with three providers:

- **Provider** (react-redux) — gives every component access to the Redux store.
- **PersistGate** — waits for the saved state to load before showing the app.
- **Toaster** — enables the popup notifications.

It also exports **BASE_URL**, the address of the backend, used by every API call.

```
export const BASE_URL = "http://localhost:8080"
```

App.js — routing & socket

App.js defines the pages with **React Router**:

Path	Component	Purpose
/	HomePage	The chat screen (must be logged in).
/login	Login	Sign in.
/signup	Signup	Create an account.

It also opens the **Socket.IO** connection when you are logged in, and listens for the **getOnlineUsers** event to know who is online. When you log out, it closes the socket.

6. Frontend — Pages

Login

- Collects username & password; toggles password visibility with an eye icon.
- Calls POST `/api/v1/user/login`.
- On success: saves you with **dispatch(setAuthUser(...))** and navigates to `/`.
- On failure: shows a red toast 'Incorrect username or password'.

Signup

- Collects full name, username, password, confirm password, and gender (single-select).
- Calls POST `/api/v1/user/register`; on success navigates to `/login`.
- Left side shows the welcome panel (AuthWelcome) describing the app's features.

HomePage

- Guards the route: if not logged in (no `authUser`), it redirects to `/login`.
- Lays out the **Sidebar** (40% width) and the **MessageContainer** (60%).

7. Frontend — Components

AuthWelcome — The blue feature panel on the Login/Signup pages (brand + highlights).

Sidebar — Your profile header, search, the user list, and the red Logout button.

OtherUsers — Fetches and lists all other users (uses the `useGetOtherUsers` hook).

OtherUser — A single user row: avatar, name, and online/offline status; click to open that chat.

MessageContainer — The right side: chat header (with the friend's profile), messages, and the input.

Messages — Scrolls and renders all messages; wires up live updates.

Message — One message bubble — supports text, an image, or a file attachment.

SendInput — The composer: text box, attach (paperclip) button, and send button.

ProfileModal — Popup showing a user's details (username, gender, joined date, status). Your own profile is editable.

How a click becomes a chat



8. Frontend — Utilities & Hooks

utils/avatar.js

- **defaultAvatar(user)** — builds a colored circle with the user's initials (an SVG), so an avatar always shows.
- **avatarSrc(user)** — uses an uploaded photo if present, otherwise the initials avatar.
- **avatarOnError(user)** — if a photo fails to load, falls back to the initials avatar.

utils/helpers.js

- **formatJoined(iso)** — turns a stored date into a friendly '25 May 2026'.
- **isUserOnline(user, onlineUsers)** — true only if the user is connected AND hasn't hidden their status.

Hooks (reusable data logic)

Hook	What it does
useGetOtherUsers	On load, GET /user and store the list with setOtherUsers.
useGetMessages	When you open a chat, GET /message/:id and store with setMessages.
useGetRealTimeMessage	Listens to the socket 'newMessage' event and appends it with setMessages.

A 'hook' is a reusable function (name starts with 'use') that adds behavior to a component.

9. Redux State Management (full flow)

Redux is a central box (the **store**) that holds shared data. Components **read** it with **useSelector** and **change** it by **dispatching** actions. When the data changes, the components using it re-render automatically.

The golden loop

Component	dispatch(action)	Reducer updates store	useSelector re-renders
-----------	------------------	-----------------------	------------------------

Your store

```
store = {  
  user:    { authUser, otherUsers, selectedUser, onlineUsers },  
  message: { messages },  
  socket:  { socket }  
}
```

The three slices

Slice	State	Actions
userSlice	authUser, otherUsers, selectedUser, onlineUsers	setAuthUser, setOtherUsers, setSelectedUser, setOnlineUsers
messageSlice	messages	setMessages
socketSlice	socket	setSocket

Key terms defined

Store — The single object holding all shared state.

Slice — A named part of the store plus the functions that change it.

Action — A plain instruction describing a change (e.g. setMessages).

Reducer — The function that applies an action to the state.

dispatch — How a component sends an action to the store.

useSelector — How a component reads a value from the store.

redux-persist — Saves the store to localStorage so a refresh keeps your data.

10. Backend — Server & Middleware

index.js

- Loads env vars with dotenv and connects to MongoDB.
- Registers global **middleware**: express.json (parse bodies), cookieParser (read cookies), cors (allow the frontend).
- Mounts the routers under /api/v1/user and /api/v1/message.
- Starts the combined HTTP + Socket.IO server with server.listen.

What is middleware?

Middleware is a function that runs *between* the request arriving and the controller answering. It can inspect or modify the request, then call **next()** to continue. Examples here: JSON parsing, cookie reading, CORS, and the authentication check.

isAuthenticated middleware

```
const token = req.cookies.token;
if(!token) return res.status(401).json({message:"User not authenticated."});
const decode = jwt.verify(token, SECRET);
req.id = decode.userId;    // controllers now know who you are
next();
```

11. Backend — Routes & Controllers

Routes map URLs to controllers

```
// userRoute.js
router.post("/register", register)
router.post("/login", login)
router.get("/logout", logout)
router.post("/profile/update", isAuthenticated, updateProfile)
router.get("/", isAuthenticated, getOtherUsers)

// messageRoute.js
router.post("/send/:id", isAuthenticated, sendMessage)
router.get("/:id", isAuthenticated, getMessage)
```

User controller functions

register — Validates input, hashes the password (bcrypt), creates the user, returns success.

login — Checks the password, signs a JWT, sets it as an httpOnly cookie, returns your profile.

logout — Clears the cookie.

updateProfile — Updates your name, photo, gender, or status visibility.

getOtherUsers — Returns every user except you (without passwords).

Message controller functions

sendMessage — Finds or creates the conversation, uploads any attachment, creates the Message, saves it, and pushes it live via Socket.IO.

getMessage — Finds the conversation between the two users and returns all its messages (populated).

12. Database Design (models & relationships)

Data is stored in **MongoDB** as documents. **Mongoose schemas** define the shape of each document.

User

Field	Type	Notes
fullName	String	Required.
username	String	Required, unique.
password	String	Required, stored hashed.
profilePhoto	String	Photo URL or uploaded image.
gender	String	'male' or 'female'.
showStatus	Boolean	If false, others can't see your status.
createdAt / updatedAt	Date	Added automatically (timestamps).

Message

Field	Type	Notes
senderId	ObjectId → User	Who sent it.
receiverId	ObjectId → User	Who receives it.
message	String	The text (may be empty if media only).
image	String	Image URL, if any.
fileUrl / fileName	String	File link & name, if any.

Conversation

Field	Type	Notes
participants	[ObjectId → User]	The two users.
messages	[ObjectId → Message]	All messages in this chat.

Relationships

User A	Conversation (A,B)	[Messages]	User B
--------	--------------------	--------------	--------

A **Conversation** connects two users and references a list of **Message** documents. The **ObjectId** fields are **references** — like a pointer to another document — which Mongoose can **populate** (replace the id with the full document) when loading a chat.

13. Authentication & Security

Registration

Validate	bcrypt hash password	Create user	Success
----------	----------------------	-------------	---------

Login

Find user	bcrypt.compare	Sign JWT	Set httpOnly cookie
-----------	----------------	----------	---------------------

Why these choices are secure

Hashing (bcrypt) — Passwords are scrambled one-way. Even if the DB leaks, real passwords aren't exposed.

JWT — A signed token the server can verify without storing sessions.

httpOnly cookie — JavaScript can't read it, which protects the token from many attacks (XSS).

isAuthenticated — Every protected route verifies the cookie before running.

Frontend vs backend 'logged in'

Question	Where	How
Show chat or login page?	Frontend	Is Redux authUser not null?
Is this API call allowed?	Backend	Is the JWT cookie valid?

14. Real-Time Messaging (Socket.IO)

Socket.IO keeps an always-open connection so the server can push data to the browser the instant it happens — no refresh, no polling.

On the server (socket/socket.js)

- When a user connects, save **userId** → **socketId** in a map (userSocketMap).
- Broadcast **getOnlineUsers** (all online ids) to everyone.
- **getReceiverSocketId(id)** finds a specific user's socket so a message goes only to them.
- On disconnect, remove the user and broadcast the updated online list.

On the client

- App.js opens the socket when you log in and stores it in Redux (setSocket).
- It listens for **getOnlineUsers** → setOnlineUsers (green dots).
- useGetRealTimeMessage listens for **newMessage** → setMessages (live bubble).

Sender sends	Server saves	emit newMessage	Receiver sees instantly
--------------	--------------	-----------------	-------------------------

15. Media Uploads (images & files)

Attachments (images, PDFs, docs) are handled like this:



Step by step

- In **SendInput**, the paperclip opens a file picker; the file is read into a base64 **data URI**.
- It's sent to **sendMessage** as 'media' (plus 'fileName').
- The server's **uploadMedia** helper sends it to **Cloudinary** and stores the returned URL in the message (image or fileUrl).
- The **Message** component shows images inline and files as a downloadable chip.

Smart fallback

If Cloudinary keys are not set, **uploadMedia** simply stores the data URI directly, so sending still works in development. Add the keys (Section 19) to switch to real hosting — no code change needed.

base64 / data URI — A way to embed a file's bytes as a text string the browser can read directly.

Cloudinary — A cloud service that stores media and returns a fast public URL.

16. Complete API Reference

Method	Endpoint	Auth	Body / Params	Returns
POST	/api/v1/user/register	No	fullName, username, password, confirmPassword, gender	success message
POST	/api/v1/user/login	No	username, password	your profile + sets cookie
GET	/api/v1/user/logout	No	—	clears cookie
POST	/api/v1/user/profile/update	Yes	fullName, profilePhoto, gender, showStatus	updated user
GET	/api/v1/user/	Yes	—	list of other users
POST	/api/v1/message/send/:id	Yes	message, media, fileName	the new message
GET	/api/v1/message/:id	Yes	id = other user	all messages

Auth = requires a valid login cookie (the isAuthenticated middleware).

17. End-to-End Data Flows

A) Registration

Fill form	POST register	Hash + save	Go to login
-----------	---------------	-------------	-------------

B) Login

POST login	Verify + JWT cookie	setAuthUser	Show chat
------------	---------------------	-------------	-----------

C) Loading the app

- useGetOtherUsers → GET /user → setOtherUsers (sidebar fills).
- Socket connects → setSocket; server sends online list → setOnlineUsers.

D) Opening a chat & messaging

setSelectedUser	GET messages	Type/attach	POST send	socket newMessage
-----------------	--------------	-------------	-----------	-------------------

E) Editing profile / hiding status

Open profile	Edit + Save	POST profile/update	setAuthUser
--------------	-------------	---------------------	-------------

F) Logout

Click Logout	GET logout	Clear cookie	setAuthUser(null)
--------------	------------	--------------	-------------------

18. Feature Walkthroughs

Online status privacy

Each user has **showStatus**. If you turn it off in your profile, **isUserOnline** returns false for you on everyone else's screen — so no one sees your green dot, even though you're connected. You still see your own real status.

Default avatars

If a user hasn't uploaded a photo, the app generates a colored circle with their initials, so the UI never shows a blank avatar.

Profile (view & edit)

- Click your own profile → view details → Edit → change name, gender, photo (from device), status.
- Click a friend's header or the info icon → view their photo, username, gender, and joined date.

Persisted login

redux-persist saves the store to localStorage, so refreshing the page keeps you logged in and remembers the open chat.

19. Environment Variables

Backend (backend/.env)

Variable	Purpose
PORT	Port the server runs on (e.g. 8080).
MONGO_URI	MongoDB connection string.
JWT_SECRET_KEY	Secret used to sign/verify JWT tokens.
CLOUDINARY_CLOUD_NAME	Cloudinary account name (optional).
CLOUDINARY_API_KEY	Cloudinary API key (optional).
CLOUDINARY_API_SECRET	Cloudinary API secret (optional).

Frontend

BASE_URL in src/index.js points to the backend (http://localhost:8080 in development).

Cloudinary variables are optional — without them, media is stored inline as a fallback.

20. Deployment Guide

Recommended setup

Layer	Platform	Notes
Frontend	Vercel / Netlify	Build with 'npm run build'.
Backend	Render / Railway	Start with 'npm start'.
Database	MongoDB Atlas	Allow access from anywhere (0.0.0.0/0).
Media	Cloudinary	Add the 3 Cloudinary env vars.

Things to change for production

- Point **BASE_URL** (frontend) to the deployed backend URL (https).
- Update **CORS origin** (backend) to the deployed frontend URL.
- Update the Socket.IO cors origin to the deployed frontend URL.
- Cookies: use secure + sameSite 'none' so they work across domains (HTTPS).

21. Annotated Source Code

This chapter walks through the most important files with the real code and a short explanation of what each part does. Read the explanation first, then the code.

Frontend · src/index.js

Boots the React app and wires Redux + persistence + toasts. Exports the backend address.

```
export const BASE_URL = "http://localhost:8080";

let persistor = persistStore(store);
root.render(
  <Provider store={store}>
    <PersistGate loading={null} persistor={persistor}>
      <App />
      <Toaster />
    </PersistGate>
  </Provider>
);
```

Provider shares the store; PersistGate restores saved state; Toaster enables notifications.

Frontend · src/App.js (routing + socket)

Defines the three routes and opens the Socket.IO connection when logged in.

```
const router = createBrowserRouter([
  { path: "/", element: <HomePage/> },
  { path: "/signup", element: <Signup/> },
  { path: "/login", element: <Login/> },
]);

useEffect(() => {
  if (authUser) {
    const s = io(BASE_URL, { query: { userId: authUser._id } });
    dispatch(setSocket(s));
    s.on("getOnlineUsers", (u) => dispatch(setOnlineUsers(u)));
    return () => s.close();
  } else if (socket) { socket.close(); dispatch(setSocket(null)); }
}, [authUser]);
```

Frontend · redux/store.js

Combines the three slices and wraps them with redux-persist.

```
const rootReducer = combineReducers({
  user: userReducer,
  message: messageReducer,
  socket: socketReducer,
});
const persistedReducer = persistReducer({key:"root", version:1, storage}, rootReducer);
const store = configureStore({ reducer: persistedReducer, /* ignore socket in serialize check */ });
```

Frontend · redux/userSlice.js

Holds the core user state and the setters that change it.

```
initialState: { authUser:null, otherUsers:null, selectedUser:null, onlineUsers:null }

reducers: {
  setAuthUser: (s,a)=> { s.authUser = a.payload; },
  setOtherUsers: (s,a)=> { s.otherUsers = a.payload; },
  setSelectedUser: (s,a)=> { s.selectedUser = a.payload; },
  setOnlineUsers: (s,a)=> { s.onlineUsers = a.payload; },
}
```

Frontend · redux/messageSlice.js & socketSlice.js

```
// messageSlice
initialState: { messages:null }
setMessages: (s,a)=> { s.messages = a.payload; }

// socketSlice
initialState: { socket:null }
setSocket: (s,a)=> { s.socket = a.payload; }
```

Frontend · hooks

Three small hooks fetch data and push it into Redux.

```
// useGetOtherUsers – on mount
const res = await axios.get(`${BASE_URL}/api/v1/user`);
dispatch(setOtherUsers(res.data));

// useGetMessages – when selectedUser changes
const res = await axios.get(`${BASE_URL}/api/v1/message/${selectedUser._id}`);
dispatch(setMessages(res.data));

// useGetRealTimeMessage – live
socket?.on("newMessage", (m)=> dispatch(setMessages([...messages, m])));
```

Frontend · components/SendInput.jsx (sending text + media)

```
const reader = new FileReader();
reader.onloadend = () => { setMedia(reader.result); setFileName(file.name); };
reader.readAsDataURL(file);

await axios.post(`${BASE_URL}/api/v1/message/send/${selectedUser._id}`,
  { message, media, fileName }, { withCredentials:true });
```

Frontend · components/Message.jsx (rendering a bubble)

```
{message.image && <img src={message.image} .../>}
{message.fileUrl && <a href={message.fileUrl} download>{message.fileName}</a>}
{message.message && <div className="bubble">{message.message}</div>}
```

Backend · middleware/isAuthenticated.js

Guards protected routes by verifying the JWT cookie.

```
const token = req.cookies.token;
if(!token) return res.status(401).json({message:"User not authenticated."});
const decode = jwt.verify(token, process.env.JWT_SECRET_KEY);
req.id = decode.userId;
next();
```

Backend · controllers/userController.js — register & login

```
// register
const hashed = await bcrypt.hash(password, 10);
await User.create({ fullName, username, password:hashed, gender, profilePhoto });

// login
const ok = await bcrypt.compare(password, user.password);
if(!ok) return res.status(400).json({message:"Incorrect username or password"});
const token = jwt.sign({userId:user._id}, SECRET, {expiresIn:"1d"});
res.cookie("token", token, { httpOnly:true, sameSite:"strict" }).json(user);
```

Backend · controllers/messageController.js — sendMessage

```
const { message, media, fileName } = req.body;
let image="", fileUrl="", fName="";
if (media) {
  const url = await uploadMedia(media); // Cloudinary or inline fallback
  if (media.startsWith("data:image")) image = url;
  else { fileUrl = url; fName = fileName; }
}
const newMessage = await Message.create({ senderId, receiverId, message, image, fileUrl, fileName:fName });
conversation.messages.push(newMessage._id); await conversation.save();

const sid = getReceiverSocketId(receiverId);
if (sid) io.to(sid).emit("newMessage", newMessage); // live delivery
```

Backend · socket/socket.js

```
const userSocketMap = {}; // userId -> socketId
io.on("connection", (socket) => {
  const userId = socket.handshake.query.userId;
  if (userId) userSocketMap[userId] = socket.id;
  io.emit("getOnlineUsers", Object.keys(userSocketMap));
  socket.on("disconnect", () => {
    delete userSocketMap[userId];
    io.emit("getOnlineUsers", Object.keys(userSocketMap));
  });
});
export const getReceiverSocketId = (id) => userSocketMap[id];
```

Backend · config/cloudinary.js (upload with fallback)

```
export const uploadMedia = async (dataUri) => {
  const c = await getCloudinary(); // null if not configured
  if (!c) return dataUri; // fallback: store inline
  const res = await c.uploader.upload(dataUri, { resource_type:"auto" });
  return res.secure_url; // hosted URL
};
```

22. Glossary — every term defined

API — A set of URLs the frontend calls to get or change data.

Axios — A library for making HTTP requests from the browser.

Backend — The server side: logic, database, and APIs.

base64 / data URI — A file encoded as a text string the browser can use directly.

bcrypt — A library that hashes passwords securely.

Component — A reusable, self-contained piece of UI in React.

Controller — A backend function with the logic for a specific request.

Cookie — A small piece of data the browser stores and sends back to the server.

CORS — A rule controlling which websites may call your API.

Dispatch — Sending an action to the Redux store to change state.

Frontend — The client side the user sees and interacts with.

Hashing — One-way scrambling of data (used for passwords).

Hook — A reusable function (use...) that adds behavior to a component.

HTTP — The request/response protocol used for normal web calls.

httpOnly cookie — A cookie JavaScript cannot read — safer for tokens.

JWT — A signed token that proves who you are.

Middleware — Code that runs between a request and its controller.

MongoDB — A NoSQL database storing JSON-like documents.

Mongoose — A tool to define schemas and query MongoDB.

ObjectId — MongoDB's unique id; used to reference other documents.

ODM — Object Data Modelling — maps code objects to DB documents.

populate — Replacing a referenced id with the full document.

Props — Inputs passed into a React component.

Reducer — A function that updates state in response to an action.

Redux — A central store for app-wide shared state.

redux-persist — Saves/restores the Redux store from browser storage.

Route — A mapping from a URL to code (page on frontend, controller on backend).

Schema — The defined shape/fields of a database document.

Selector (useSelector) — Reads a value from the Redux store.

Slice — A named section of the Redux store + its reducers.

Socket.IO — Real-time two-way communication over WebSockets.

State — Data that can change over time and drives the UI.

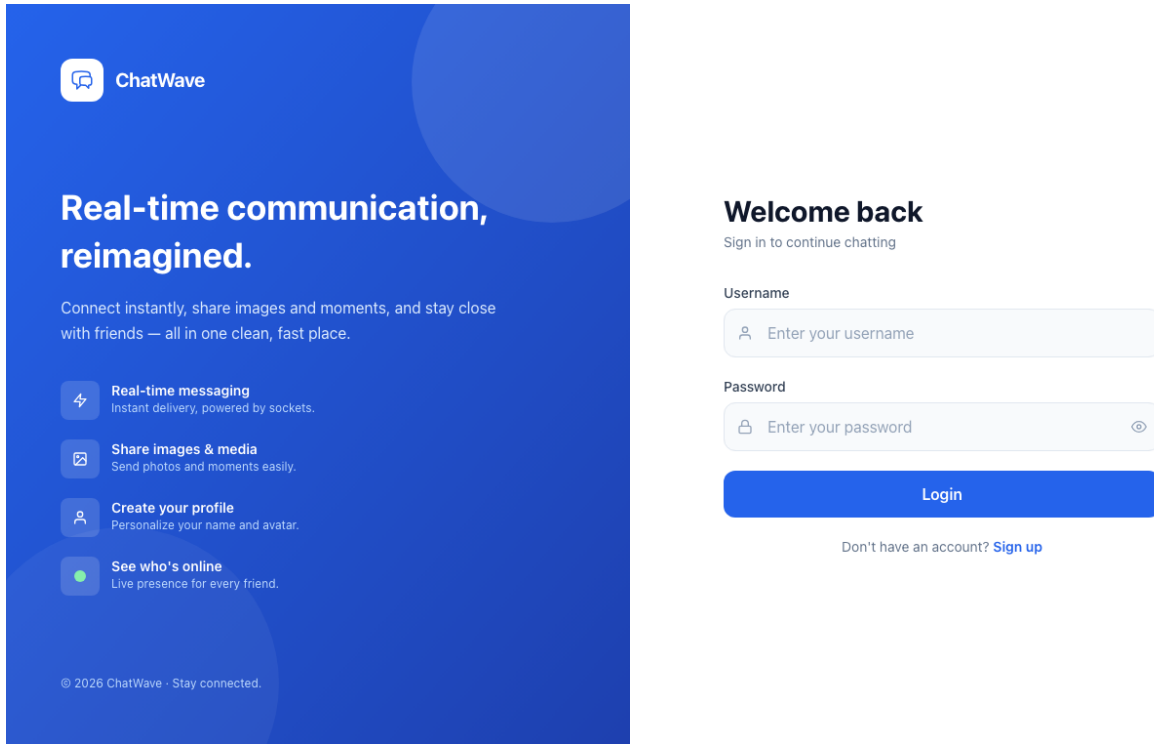
Token — A string proving identity (here, a JWT).

WebSocket — A persistent connection allowing instant two-way data.

The end. You now have a complete picture of ChatWave — from a button click in the browser, through Redux and the API, into MongoDB, and back out live over Socket.IO.

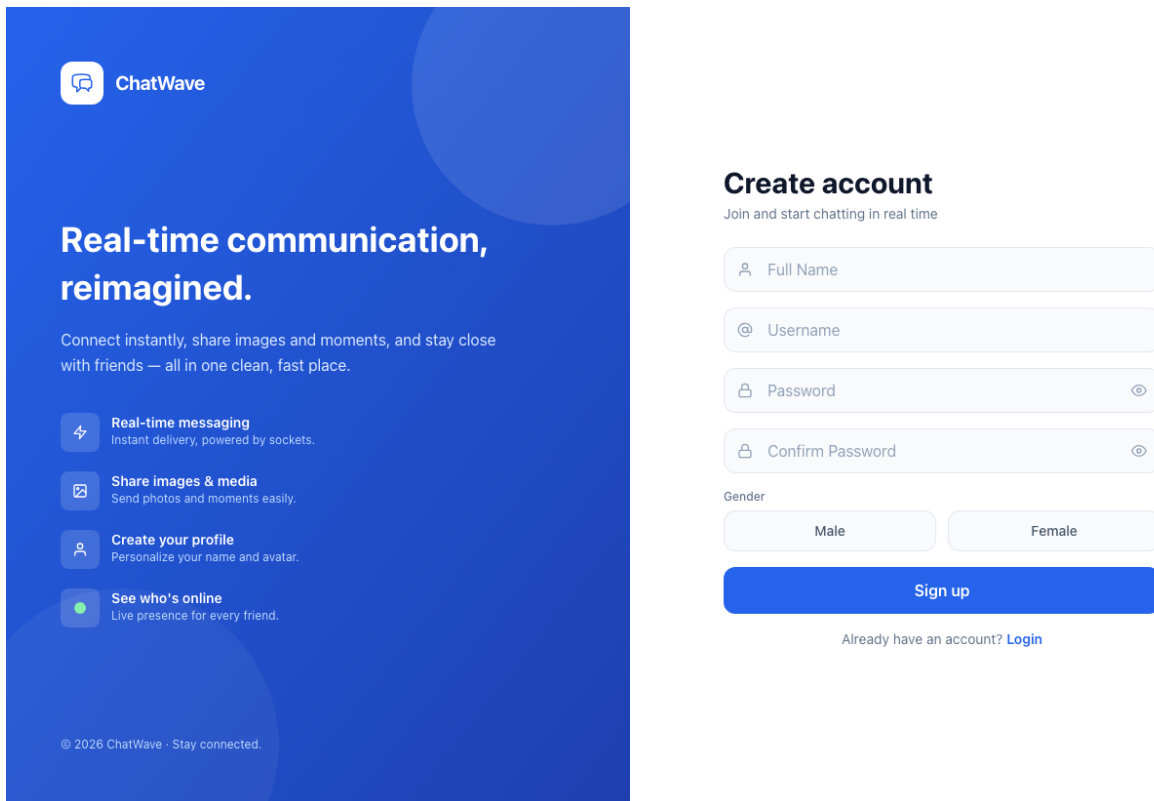
23. UI Gallery

Real screenshots of the finished application.



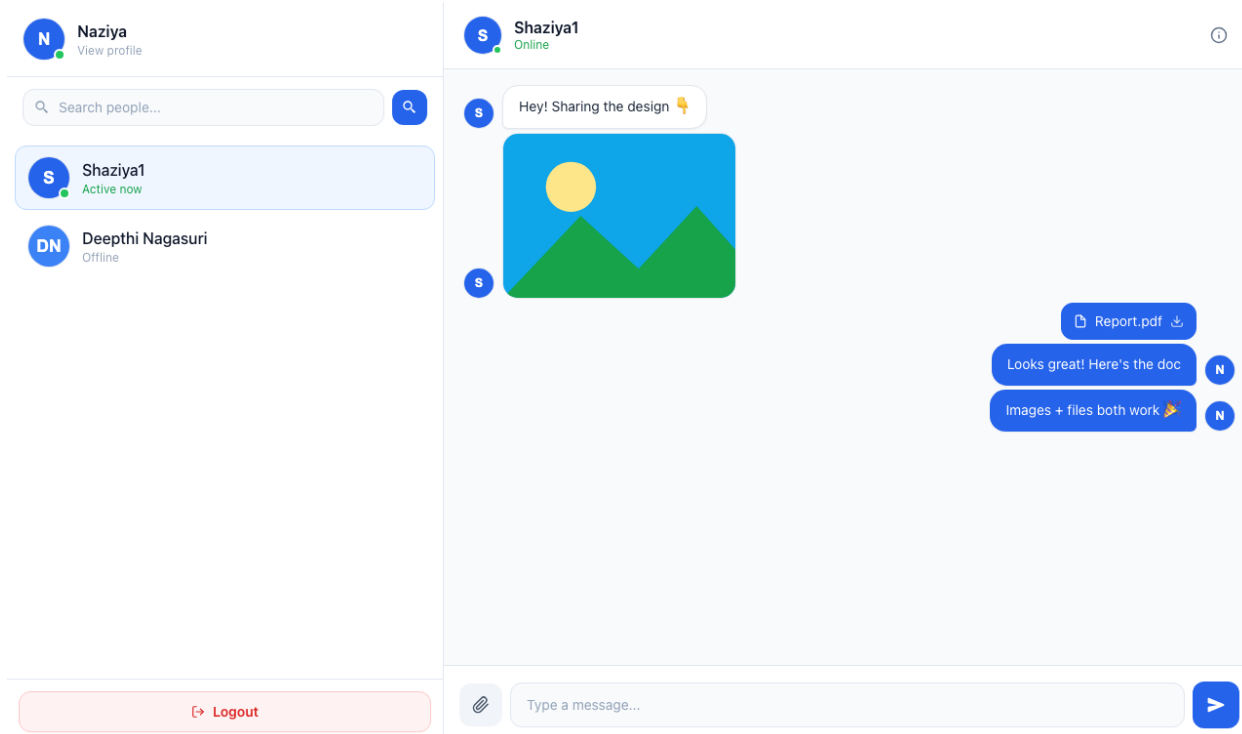
The login screen for ChatWave features a blue background with a white chat bubble icon and the text "ChatWave" in the top left. The main heading is "Real-time communication, reimagined." followed by a subheading "Connect instantly, share images and moments, and stay close with friends — all in one clean, fast place." Below this are four feature cards: "Real-time messaging" (Instant delivery, powered by sockets.), "Share images & media" (Send photos and moments easily.), "Create your profile" (Personalize your name and avatar.), and "See who's online" (Live presence for every friend.). At the bottom left is the copyright notice "© 2026 ChatWave · Stay connected." On the right, the "Welcome back" section includes the text "Sign in to continue chatting" and a "Username" input field with a placeholder "Enter your username". Below that is a "Password" input field with a placeholder "Enter your password" and a show/hide toggle. A blue "Login" button is positioned below the password field. At the bottom right, there is a link "Don't have an account? Sign up".

Login — full-width two-column layout with welcome panel and password show/hide.



The signup screen for ChatWave features a blue background with a white chat bubble icon and the text "ChatWave" in the top left. The main heading is "Real-time communication, reimagined." followed by a subheading "Connect instantly, share images and moments, and stay close with friends — all in one clean, fast place." Below this are four feature cards: "Real-time messaging" (Instant delivery, powered by sockets.), "Share images & media" (Send photos and moments easily.), "Create your profile" (Personalize your name and avatar.), and "See who's online" (Live presence for every friend.). At the bottom left is the copyright notice "© 2026 ChatWave · Stay connected." On the right, the "Create account" section includes the text "Join and start chatting in real time" and four input fields: "Full Name", "Username", "Password" (with a show/hide toggle), and "Confirm Password" (with a show/hide toggle). Below these is a "Gender" section with "Male" and "Female" radio buttons. A blue "Sign up" button is positioned below the gender section. At the bottom right, there is a link "Already have an account? Login".

Signup — name, username, password, gender select, with the same welcome panel.



Chat — wider sidebar (40/60), initials avatars, online dots, image + file messages, red logout.

Avatars are generated from initials when no photo is uploaded, so they always render.

24. FAQ — common questions

Q: How does the app know I'm logged in?

The frontend checks the Redux value `authUser` (not null = logged in). The backend checks the JWT cookie via the `isAuthenticated` middleware. The frontend decides what to show; the backend decides what to allow.

Q: Why do I stay logged in after refresh?

`redux-persist` saves the Redux store to the browser's `localStorage` and restores it on startup, so `authUser` is still there.

Q: Why did a friend appear Offline while online?

That user turned OFF 'Show online status' in their profile (`showStatus=false`). `isUserOnline` hides their dot from others. They can turn it back on in Edit Profile.

Q: Where are uploaded images stored?

If Cloudinary keys are set, on Cloudinary (only the URL is saved in MongoDB). If not, the file is stored inline as a data URI so it still works in development.

Q: How do messages arrive instantly?

Socket.IO keeps a live connection. When you send, the server emits a 'newMessage' event to the receiver's socket, and their UI appends it immediately.

Q: How are passwords kept safe?

They are hashed with `bcrypt` before saving. The original password is never stored, and the token lives in an `httpOnly` cookie JavaScript can't read.

Q: How do I add a new API endpoint?

Add a route in `routes/`, write a controller function in `controllers/`, and (if it needs login) put `isAuthenticated` in front of it.

Q: How do I change the theme color?

The UI uses Tailwind 'blue-600' as the accent. Search the components for 'blue-600' / 'blue-700' and replace with your color.

25. Getting Started — Setup & Run

Prerequisites

- **Node.js** (v16+) and npm installed.
- A **MongoDB** database — local, or a free MongoDB Atlas cluster.
- (Optional) a **Cloudinary** account for media hosting.

1) Backend setup

```
cd backend
npm install
# create backend/.env :
PORT=8080
MONGO_URI=your_mongodb_connection_string
JWT_SECRET_KEY=any_long_random_string
# optional:
CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=...
CLOUDINARY_API_SECRET=...

npm run dev    # starts the server on PORT
```

2) Frontend setup

```
cd frontend
npm install
npm start      # opens http://localhost:3000
```

3) First run

- Open <http://localhost:3000> — you'll land on the Login page.
- Click **Sign up**, create an account, then log in.
- Open a second browser (or incognito) and sign up as another user to test real-time chat.

The frontend talks to the backend via `BASE_URL` in `src/index.js` (<http://localhost:8080> by default).

26. Testing the Application

Manual test checklist

Area	What to check
Signup	All fields required; gender must be picked; redirects to login.
Login	Wrong password shows a red toast; correct login opens chat.
Presence	A second logged-in user appears 'Active now' with a green dot.
Messaging	Text sends instantly on both sides (real-time).
Attachments	An image shows inline; a file shows a download chip.
Profile	Edit name/gender/photo/status saves and updates the UI.
Status privacy	Turning status off hides your dot from others.
Persistence	Refresh keeps you logged in and on the same chat.
Logout	Returns to login; protected pages redirect away.

Two-user test (real-time)

- Browser A: log in as User 1. Browser B (incognito): log in as User 2.
- Open each other's chat; send messages both ways — they should appear without refreshing.
- Watch the green online dots appear/disappear as you log in/out.

27. Troubleshooting & Common Errors

CORS error in the browser console

The backend only allows a specific origin. Make sure the CORS origin in index.js and socket.js matches the frontend URL (http://localhost:3000 in dev).

Login works but you're logged out on refresh / 401s

Cookie issue. In development, axios must send credentials (withCredentials:true) and the cookie sameSite must allow it. In production over HTTPS use secure:true + sameSite:'none'.

Cannot connect to MongoDB

Check MONGO_URI. On Atlas, allow your IP (or 0.0.0.0/0) under Network Access.

Messages don't appear in real time

Check the socket connects to the correct BASE_URL and that the userId query is passed. Confirm the server logs a connection.

Image upload seems large / slow

Without Cloudinary, images are stored inline (base64) which is heavy. Add Cloudinary keys to host them and keep the database light.

Avatar shows blank

Handled: the app falls back to a generated initials avatar. If you still see blank, hard-refresh to clear the old cached bundle.

Port already in use

Another process uses the port. Stop it, or change PORT (backend) / use a different port for the frontend.

28. Best Practices & Future Improvements

Good practices already used

- Layered backend (routes → controllers → models) keeps code organized.
- Passwords hashed; tokens in httpOnly cookies.
- Central state in Redux with a single source of truth.
- Reusable hooks and utilities (avatars, helpers).

Ideas to take it further

- **Typing indicators** and **read receipts** via extra socket events.
- **Group chats** by allowing more than two participants in a Conversation.
- **Message search** and **pagination** for long histories.
- **Push notifications** for new messages when the tab is closed.
- **Delete / edit messages** and **emoji reactions**.
- Server-side filtering of online status for stronger privacy.
- Unit/integration tests (Jest, React Testing Library, Supertest).

With these foundations and ideas, ChatWave can grow from a clean one-to-one chat into a full messaging platform.

29. Appendix — Component Reference

Key responsibilities and the core logic of each component, for quick reference.

Login.jsx

Signs the user in and stores them in Redux.

```
const res = await axios.post(`${BASE_URL}/api/v1/user/login`, user, {withCredentials:true})  
;  
dispatch(setAuthUser(res.data));  
navigate("/");  
// on error: toast.error("Incorrect username or password")
```

Signup.jsx

Creates an account; gender is a required single-select.

```
if(!user.gender) return toast.error("Please select your gender");  
const res = await axios.post(`${BASE_URL}/api/v1/user/register`, user, {withCredentials:tru  
e});  
if(res.data.success){ navigate("/login"); toast.success(res.data.message); }
```

Sidebar.jsx

Shows your profile (click to open ProfileModal), search, the user list, and logout.

```
const res = await axios.get(`${BASE_URL}/api/v1/user/logout`);  
dispatch(setAuthUser(null)); dispatch(setMessages(null));  
dispatch(setOtherUsers(null)); dispatch(setSelectedUser(null));  
navigate("/login");
```

OtherUser.jsx

One contact row; selecting it opens that conversation.

```
const isOnline = isUserOnline(user, onlineUsers); // respects status privacy
onClick={() => dispatch(setSelectedUser(user))}
```

MessageContainer.jsx

Right pane: header (opens the friend's profile), messages, and the composer.

```
const isOnline = isUserOnline(selectedUser, onlineUsers);
{selectedUser ? (<><Header/><Messages/><SendInput/></>) : (<EmptyState/>)}
{showProfile && <ProfileModal user={selectedUser} isOwn={false} .../>}
```

ProfileModal.jsx

Views any user; your own profile can be edited (name, gender, photo, status).

```
// upload photo from device -> base64
reader.onloadend = () => setPhoto(reader.result);

// save
await axios.post(`${BASE_URL}/api/v1/user/profile/update`,
  { fullName, profilePhoto:photo, gender, showStatus }, {withCredentials:true});
dispatch(setAuthUser(res.data.user));
```

AuthWelcome.jsx

The static blue feature panel shown beside the Login/Signup forms (brand + highlights). Pure presentational component — no state or API calls.

That completes the full reference. Use the Table of Contents to jump back to any topic.

30. Visual Sequence Diagrams

Each diagram shows the exact order of steps for one operation, end to end.

Register

Browser form	POST /register	bcrypt hash	User.create	201 success
--------------	----------------	-------------	-------------	-------------

Login

POST /login	bcrypt.compare	jwt.sign	Set-Cookie	setAuthUser
-------------	----------------	----------	------------	-------------

Load other users

useGetOtherUsers	GET /user	isAuthenticated	User.find	setOtherUsers
------------------	-----------	-----------------	-----------	---------------

Open a conversation

Click user	setSelectedUser	useGetMessages	GET /message/:id	setMessages
------------	-----------------	----------------	------------------	-------------

Send a text message

SendInput	POST /send/:id	Message.create	save Conversation	201 + setMessages
-----------	----------------	----------------	-------------------	-------------------

Send an image / file

Pick file	base64	POST /send/:id	uploadMedia	store URL
-----------	--------	----------------	-------------	-----------

Receive a message in real time

Server emit	newMessage	socket.on	setMessages	bubble appears
-------------	------------	-----------	-------------	----------------

Presence (online users)

Socket connect	userSocketMap	emit getOnlineUsers	setOnlineUsers	green dots
----------------	---------------	---------------------	----------------	------------

Edit profile

Open modal	Edit + Save	POST /profile/update	findByIdAndUpdate	setAuthUser
------------	-------------	----------------------	-------------------	-------------

Logout

Click Logout	GET /logout	clear cookie	setAuthUser(null)	go /login
--------------	-------------	--------------	-------------------	-----------

These ten flows cover every major action in ChatWave. Together with the chapters above, you now have a complete, end-to-end understanding of the project — frontend, backend, state, database, and real-time.